

SIMCOMP

Sistema de Comparendos de Tránsito

Arquitectura de microservicios distribuidos con Docker Swarm, Vagrant, HAProxy, Spark Analytics y observabilidad completa.

7

Microservicios

3

Nodos Swarm

5

BD PostgreSQL

4

Modos Deploy

¿Qué es SIMCOMP?

Problema

Gestión de tránsito fragmentada: personas, vehículos, infracciones y comparendos en silos sin integración real.

Solución

Plataforma distribuida con microservicios desacoplados, cada uno dueño de sus datos, comunicados vía REST/HTTP.

Principio de diseño

Sin llaves foráneas entre BDs. Referencias por UUID. Valores monetarios con DECIMAL(12,2). Health check en cada servicio.

Stack tecnológico

Backend: Node.js 22 · Express 4

Bases de datos: PostgreSQL 16 (5 clusters)

Frontend: React 19 · TypeScript · Vite · TailwindCSS

Orquestación: Docker Swarm · Vagrant · Ansible

Gateway: HAProxy (Ingress, SSL, LB)

Observabilidad: Prometheus · Grafana · cAdvisor

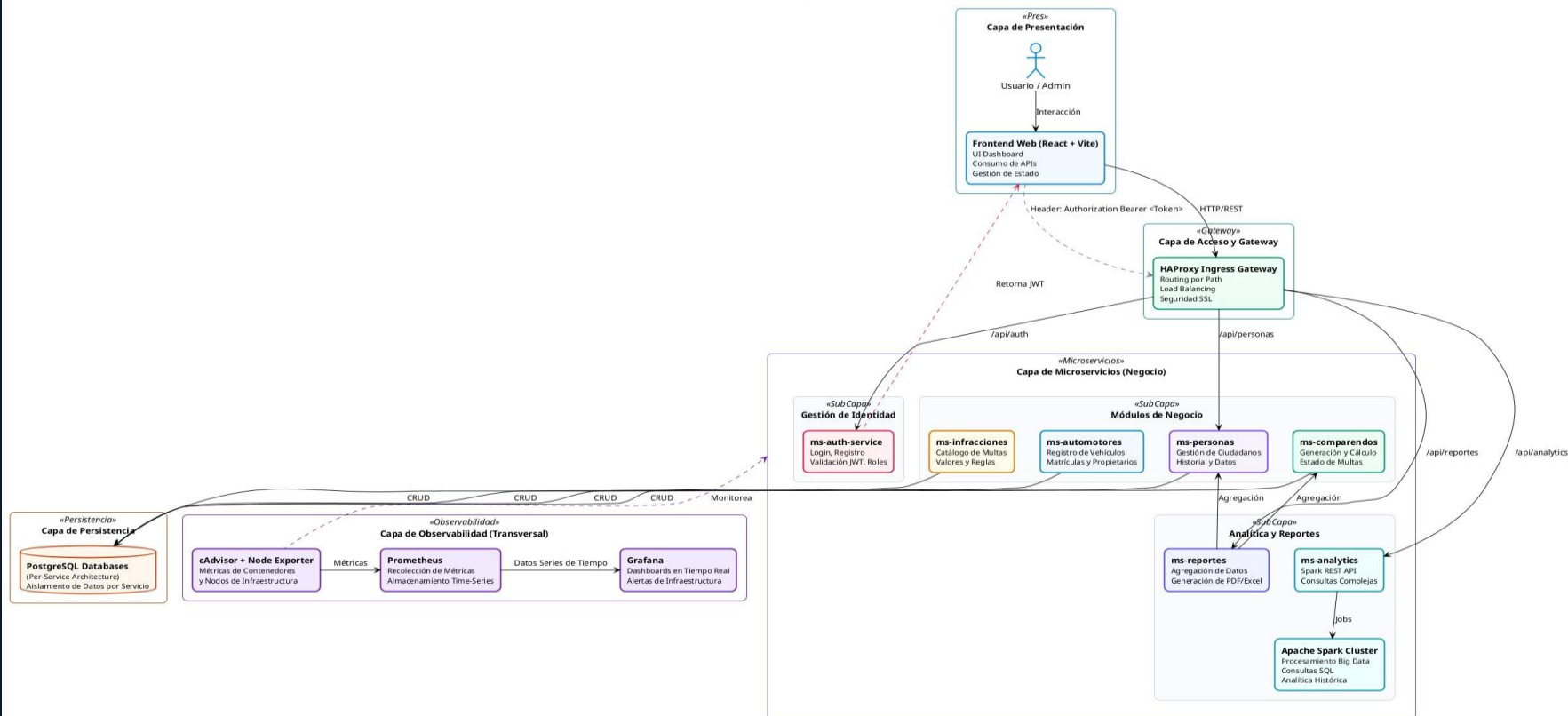
Analytics: Apache Spark · Flask API

Pruebas: Swagger/OpenAPI · JMeter

Seguridad: JWT · Docker Secrets · ACLs HAProxy

Arquitectura Lógica de Microservicios

SIMCOMP - Arquitectura Lógica de Microservicios



Microservicios Backend

:8001

ms-auth-service

Login, registro, refresh token, JWT y roles.

:8002

ms-personas

Ciudadanos, datos personales y licencias.

:8003

ms-automotores

Registro técnico del vehículo y propietario.

:8004

ms-infracciones

Catálogo oficial de multas vigentes.

:8005

ms-comparendos

Core: registra comparendos y ciclo de vida.

:8006

ms-reportes

Exportación en xls, csv y pdf.

:8010

ms-analytics

Spark REST API, consultas y analítica histórica.

Cada servicio expone /api/health · /api/docs (Swagger) · métricas Prometheus (prom-client). Sin FK físicas entre BDs — validaciones vía REST.

Persistencia: 5 Clusters PostgreSQL Independientes

Servicio	Puerto API	Base de Datos	Puerto DB	Cluster
ms-auth-service	8001	auth_db	5432	auth
ms-personas	8002	personas_db	5433	personas
ms-automotores	8003	automotores_db	5434	automotores
ms-infracciones	8004	infracciones_db	5435	infracciones
ms-comparendos	8005	comparendos_db	5436	comparendos
ms-reportes	8006	reportes_db	5432*	reportes

Dataset incluido

5000

Personas

3380

Usuarios

3400

Licencias

3000

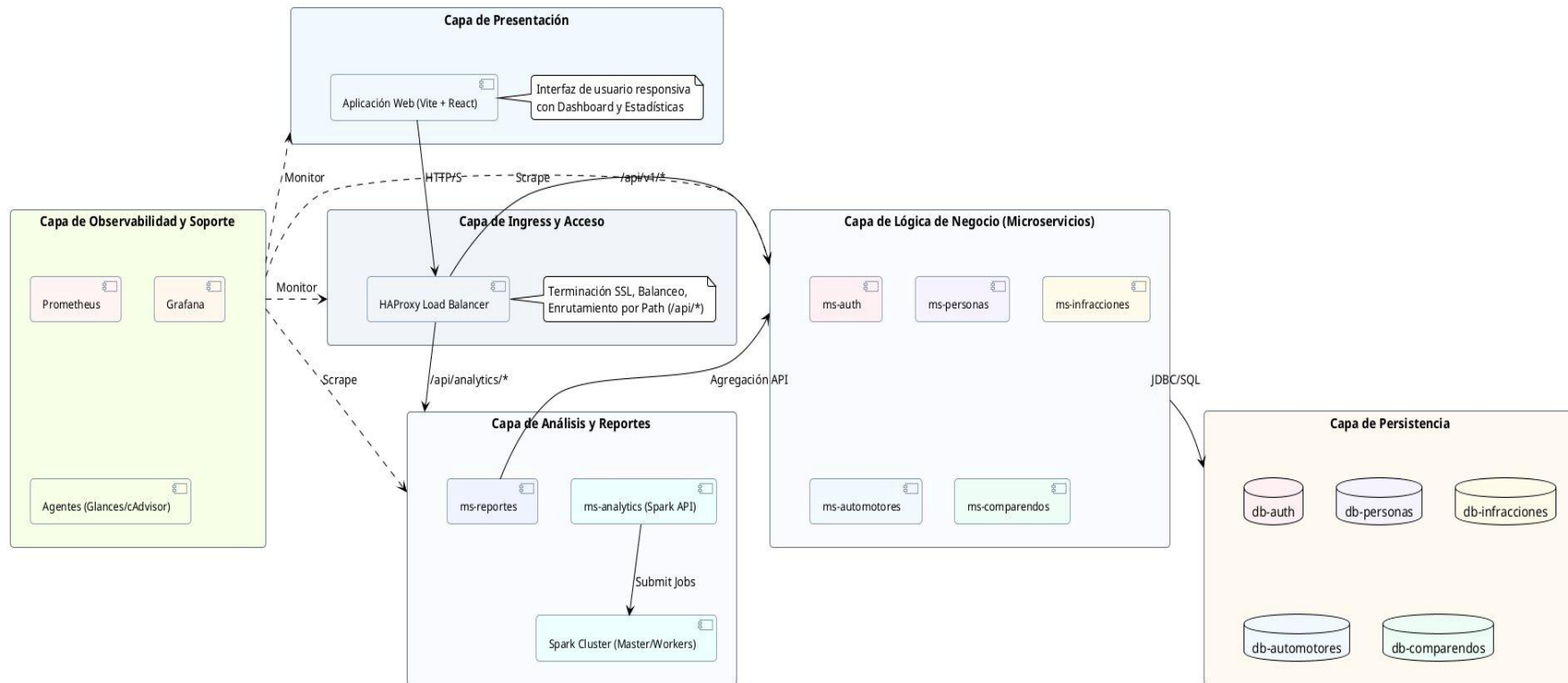
Vehículos

5000

Comparendos

Arquitectura por Capas

SIMCOMP - Diagrama de Arquitectura por Capas



4 Modos de Despliegue

01

Docker Compose Local

```
docker compose up --build -d
```

Desarrollo y demo rápida. Frontend en localhost:8080. Simula la red de producción.

02

Vagrant + Ansible (Nativo)

```
./simcomp-manager.sh → [3]
```

3 VMs: DNS (BIND9), API (PM2 + Node.js), Web (Nginx + React). Acceso en simcomp.co.

03

Vagrant + Docker Swarm

```
vagrant up + docker stack deploy
```

Cluster distribuido con HAProxy, réplicas y balanceo. Alta disponibilidad real.

04

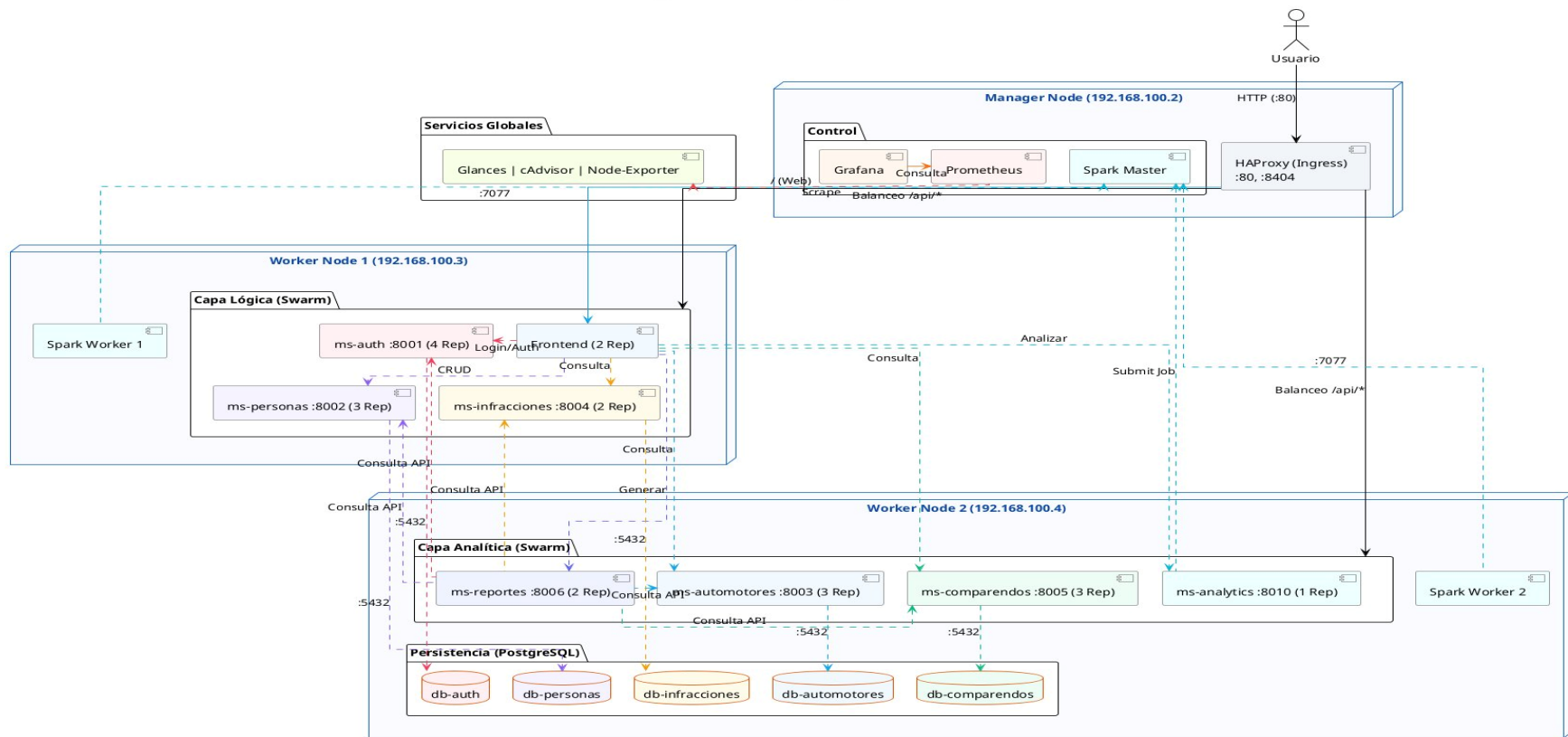
Swarm + Analytics Spark

```
./simcomp-manager.sh → Opción 3
```

Modo completo: todo lo anterior + Apache Spark Cluster para analítica histórica y JMeter.

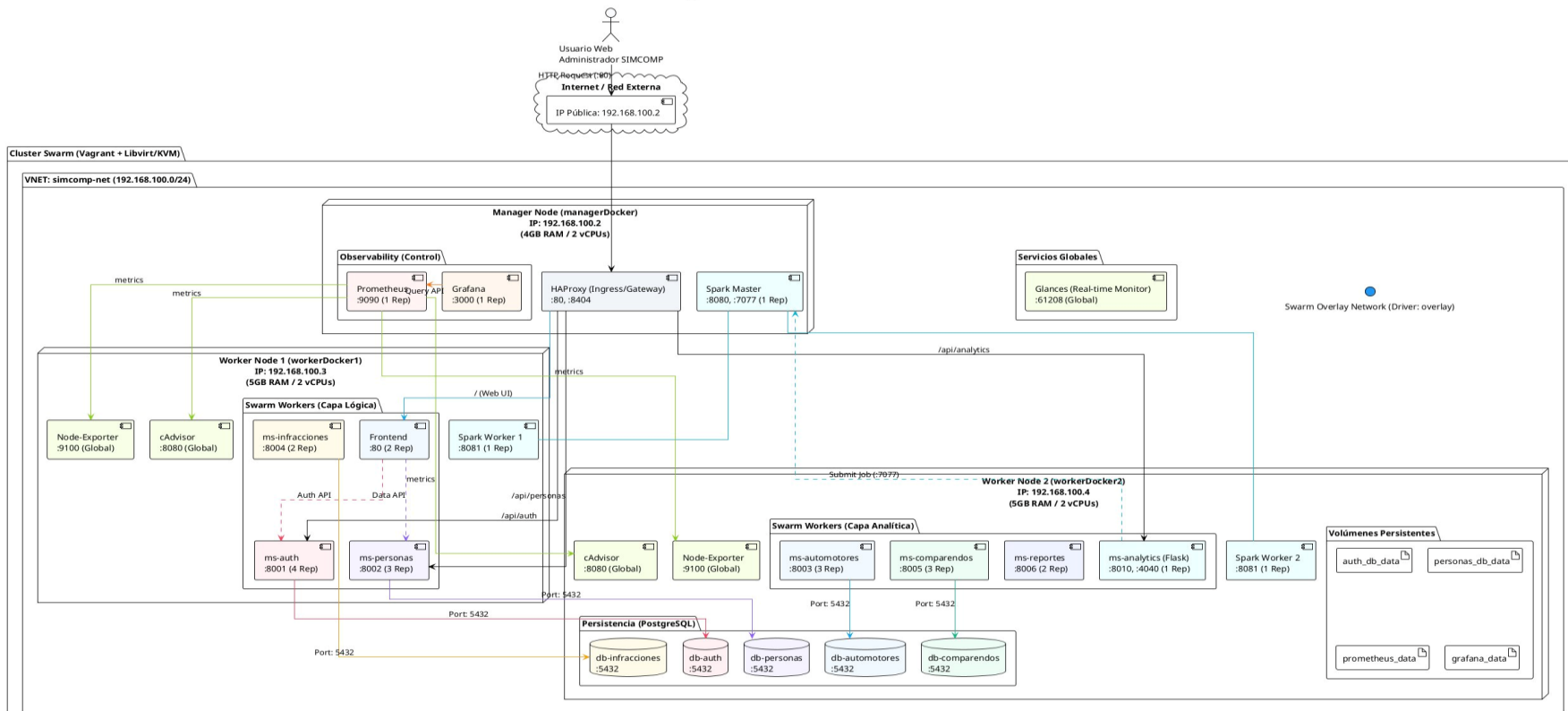
Diagrama de Despliegue — Clúster Swarm

SIMCOMP - Diagrama de Despliegue del Clúster Swarm



Infraestructura Técnica Detallada

SIMCOMP - Diagrama de Infraestructura Técnica Detallada



Cluster Swarm — 3 Nodos Virtualizados

managerDocker

192.168.100.2 · 4 GB · 2 vCPUs

Manager / Orquestador

- HAProxy (Ingress Gateway :80, :8404)
- Prometheus :9090
- Grafana :3000
- Spark Master :8080/:7077
- Glances :61208

workerDocker1

192.168.100.3 · 5 GB · 2 vCPUs

Worker — Capa Lógica

- ms-auth :8001 (4 réplicas)
- ms-personas :8002 (3 répl.)
- ms-infracciones :8004 (2 répl.)
- Frontend :80 (2 répl.)
- cAdvisor + Node-Exporter

workerDocker2

192.168.100.4 · 5 GB · 2 vCPUs

Worker — Capa Analítica

- ms-automotores :8003 (3 répl.)
- ms-comparendos :8005 (3 répl.)
- ms-reportes :8006 (2 répl.)
- ms-analytics :8010 (Flask)
- Spark Worker 2

Seguridad — Capas de Protección

JWT + Roles

Access token en memoria del cliente.
Refresh token para renovar sesión.
Gateway valida JWT antes de cada llamada API.
4 roles: admin, agente, supervisor, ciudadano.

Segmentación de red

Red overlay Docker Swarm aislada. Variables de entorno desacopladas por ambiente (.env, .env.swarm, .env.vagrant). Sin credenciales en código fuente.

Docker Secrets

Contraseñas de BD y JWT_SECRET gestionados como docker secret. Viajan cifrados, montados en /run/secrets/ dentro de los contenedores autorizados.

Soft Delete & Auditoría

Borrado lógico con campo deleted_at. Todos los registros incluyen created_at y updated_at gestionados por Sequelize.

HAProxy ACLs (Capa 7)

Bloquea acceso externo a rutas /internal/. Solo la comunicación interna entre servicios puede acceder a estas rutas.

Flujo de autenticación

Login → JWT emitido → Access token en memoria → Refresh automático
→ Si falla: redirige al login. Gateway protege cada endpoint.

Observabilidad y Monitoreo

Prometheus :9090

Recolección de métricas con DNS Service Discovery. Descubre réplicas automáticamente.

Grafana :3000

Dashboards preconfigurados. Alertas de infraestructura en tiempo real.

cAdvisor :8080/:8081

Métricas de contenedores en todos los nodos del cluster.

HAProxy Stats :8404/stats

Estado del balanceador en tiempo real. Réplicas activas / sesiones.

prom-client nativo

Métricas propias por microservicio, expuestas en /metrics.

Pipeline de Observabilidad

Microservicios

prom-client → /metrics



cAdvisor / Node-Exporter

Métricas de host y contenedores



Prometheus (DNS SD)

Scrape automático de réplicas



Grafana Dashboards

Visualización + Alertas

Frontend & Flujo de Usuario

Módulos de la SPA

Login

Autenticación y selección de rol

Dashboard

Estadísticas y métricas en tiempo real

Personas

CRUD ciudadanos y licencias

Automotores

Registro y consulta de vehículos

Infracciones

Catálogo de multas vigentes

Comparendos

Registro, pago y anulación

Reportes

Exportar xlsx / csv / pdf

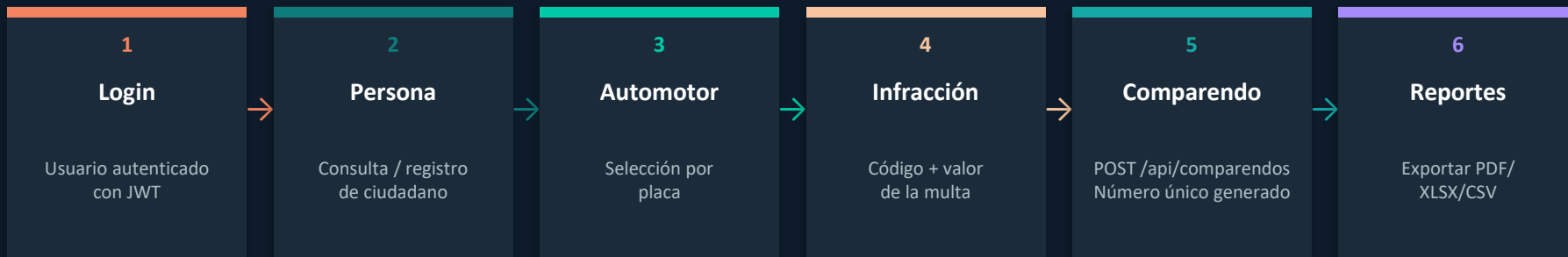
Usuarios

Administración de cuentas y roles

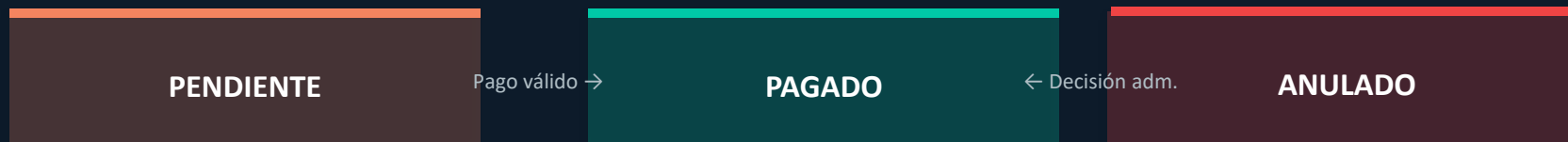
Flujo de sustentación sugerido

- 01 Login y validar roles
- 02 Dashboard y estadísticas
- 03 Consultar / registrar persona
- 04 Verificar placa / propietario
- 05 Seleccionar infracción
- 06 Crear comparendo
- 07 Pagar o anular → revisar estado

Ciclo de Vida del Comparendo



Máquina de Estados



Reglas: número único por comparendo · valor consistente con catálogo de infracciones · PATCH según estado.

Endpoints clave: GET /api/comparendos · POST /api/comparendos · PATCH /api/comparendos/:id · GET /api/reportes

Pruebas de Carga — Apache JMeter

simcomp-login.jmx

Autenticación + extracción de token JWT

simcomp-comparendos.jmx

Login + consultas al servicio de comparendos

simcomp-workflow-completo.jmx

Flujo real: Login → Personas → Vehículos → Comparendos

simcomp-estres-frontend.jmx

Optimizaciones aplicadas para soportar 500+ req/10s:

Prueba masiva de carga al servidor web

- HAProxy: maxconn 20.000 · algoritmo leastconn · Gzip habilitado · http-server-close
- ms-auth: 4 réplicas · ms-personas: 3 réplicas · NODE_ENV=production
- VMs: 4 GB Manager · 5 GB Workers · Spark Overlay Network optimizada

Ejecutar: `jmeter -n -t jmeter/simcomp-workflow-completo.jmx -l results.jtl` | Monitorear: <http://simcomp.co:8404/stats>

Accesos y URLs del Sistema

Docker Compose / Local

Frontend (HAProxy)	<code>localhost:8080</code>
HAProxy Stats	<code>localhost:8404/stats</code>
Grafana	<code>localhost:3000</code>
Prometheus	<code>localhost:9090</code>
Analytics Dashboard	<code>localhost:8010</code>
Swagger Auth	<code>localhost:8001/api/docs</code>
Swagger Comparendos	<code>localhost:8005/api/docs</code>

Vagrant / Swarm (simcomp.co)

Frontend	<code>simcomp.co</code>
HAProxy Stats	<code>simcomp.co:8404/stats</code>
Grafana	<code>simcomp.co:3000</code>
Prometheus	<code>simcomp.co:9090</code>
Spark UI	<code>simcomp.co:4040</code>
Analytics	<code>simcomp.co:8010</code>
Swagger Auth	<code>192.168.100.3:8001/api/docs</code>

SIMCOMP

Arquitectura distribuida, ejecutable y coherente con principios de dominio, seguridad y despliegue en infraestructura virtualizada.

7

Microservicios
(8001–8006 + 8010)

5

Clusters PostgreSQL
independientes

4

Modos de
despliegue

3

Nodos Swarm
virtualizados

Deyton Riascos Ortiz · Samuel Izquierdo Bonilla · Mauricio Taborda Gongora

Universidad Autónoma de Occidente — Ingeniería de Datos e Inteligencia Artificial